

Prompt Engineering for In-Context Few-Shot Learning: A Divide-and-Aggregate Approach

by

Feng Lin

A minor thesis submitted to the
School of Computer Science and Information Technology
in partial fulfillment of the requirements for the
degree of

Master of Data Science

at the
RMIT University

June 2025

Declaration

I certify that, except where due acknowledgment has been provided, this work is solely my own and has not been submitted previously, in whole or in part, for any other academic award. The content of this thesis reflects work conducted since the official commencement date of the approved research program. Any editorial contributions by third parties, whether paid or unpaid, have been acknowledged, and all ethical procedures and guidelines have been followed throughout this research.

This work has been carried out since February 2024, under the supervision of Supervisor: Dr Estrid He.



Signature

12/06/2025

Date

Name: Feng Lin

School of Computer Science and Information Technology

RMIT University

June 12, 2025

Acknowledgments

I would like to express my sincere gratitude to my supervisor, Dr Estrid He, for her valuable guidance and continuous support throughout this journey. Her initiative and insightful perspectives shaped and guided this project. I am especially thankful to Dr Tabinda Sarwar for her academic knowledge sharing, directions and feedback from the beginning of the project. I would also like to thank Dr Hai Dong, the course coordinator, for his prompt assistance with course-related matters. I also wish to thank Dr Patrick Taylor, Cloud Application Specialist at RMIT AWS Cloud Supercomputing (RACE), for his assistance with cloud API access, funding arrangements, and technical troubleshooting.

Summary

Generative AI has made huge progress in recent years, with large language models now used in many everyday technologies, such as chatbots and digital copilots. One thing that makes them special is their ability to “learn” from the contents provided by the user. Going through a few examples, these models can perform better in the tasks we ask of them. This thesis explores a new and general way to help these models make more accurate and reliable predictions. Traditional methods for improving performance often require careful selection or arrangement of examples, or additional complex processes. In contrast, our approach simply divides the examples into smaller groups at random and asks the model to solve each group separately. The answers are then combined through a process of calculations to get a final result. We tested this method using several popular language models and public datasets and found that it is reliable and often performs better than standard methods, without needing extra data or complicated setups.

Prompt Engineering for In-Context Few-Shot Learning: A Divide-and-Aggregate Approach

by

Feng Lin

Submitted to the School of Computer Science and Information Technology
on 12 June 2025, in partial fulfillment of the
requirements for the degree of
Master of Data Science

Abstract

Recent advances in large language models (LLMs) have brought prompt engineering to the forefront as a way to achieve strong performance through in-context learning. However, this process can be complex when aiming for optimal outputs. It often relies on manually selecting and arranging in-context examples, designing retrieval strategies, or providing step-by-step solving logics. Can we improve LLM predictions without searching for optimal examples or constructing solution steps? How can we unlock the potential of in-context learning using a simpler, more general approach? We introduce a divide-and-aggregate method for prompt engineering. Our approach randomly divides the in-context examples into groups and uses each group independently to prompt the LLM. The outputs are then combined using an entropy-weighted aggregation calculation that prioritises more informative predictions. This approach avoids the need for manual example selection, extra retrieval models, additional labelled samples or any reasoning templates. We evaluate our method on three sentiment analysis datasets using five instruction-tuned, lightweight open-source LLMs. Across all tasks, our method matches or surpasses standard baselines and remains competitive with the state-of-the-art ensemble approach, particularly in accuracy-related metrics. These results demonstrate the practicality of our approach and its potential for broader applications in prompt engineering and in-context learning.

Thesis Supervisor: Dr Estrid He
Title: Senior Lecturer

Contents

1	Introduction	17
1.1	Motivations	18
1.2	Research Questions	19
1.3	Contributions	20
1.4	Organisation	20
2	Literature Review	21
2.1	Prompting Techniques	21
2.2	Example Selection Strategies	22
2.3	Calibration and Ensembling Methods	23
3	Methodology	25
3.1	Divide-and-Aggregate Approach	25
3.2	Prompt construction	26
3.3	Entropy-Weighted Aggregation	27
4	Experiments, Results, and Analysis	29
4.1	Large Language Models	29
4.2	Datasets	30
4.2.1	Amazon Reviews	30
4.2.2	IMDB Movie Reviews	30
4.2.3	TweetEval Sentiment	31
4.3	Experiment Setup	31

4.3.1	Standard No-shot and K-shot Process	31
4.3.2	Fixed-base few-shot with extended K-shot	32
4.4	Baselines and Metrics	33
4.4.1	Baselines	33
4.4.2	Metrics	33
4.5	Results	34
4.6	Analysis	35
4.6.1	Accuracy-Related Performance	35
4.6.2	Calibration-Related Performance	35
4.6.3	Cross-Setting Performance	36
4.6.4	Cross-Model Performance	36
4.7	Discussion	36
5	Conclusion	41
A	Tables	43

List of Figures

1-1	In-context learning improves LLM output	18
1-2	Effect of different in-context example sets	19
3-1	Comparison of No-Shot and K-Shot Prompt Formats	26
3-2	Workflow of the Entropy-Weighted Prompt Aggregation Process . . .	28
4-1	MacroF1 scores for each model and dataset, across both pipelines . .	38
4-2	ECE scores for each model and dataset, across both pipelines	39

List of Tables

4.1	Summary of Evaluated Large Language Models	29
A.1	Results on Amazon Reviews Dataset (Pipeline 1)	44
A.2	Results on Amazon Reviews Dataset (Pipeline 2)	45
A.3	Results on IMDB Reviews Dataset (Pipeline 1)	46
A.4	Results on IMDB Reviews Dataset (Pipeline 2)	47
A.5	Results on TweetEval Dataset (Pipeline 1)	48
A.6	Results on TweetEval Dataset (Pipeline 2)	49

Chapter 1

Introduction

With the recent advancements, large language models (LLMs) have achieved significant progress across a range of natural language processing (NLP) tasks, including text summarisation [1], translation [2], classification [3], sentiment analysis [4], and creative generation of content such as programming code [5], songs [6], and poems [7]. LLMs now are widely used in products such as ChatGPT and Microsoft Copilot, demonstrating capabilities that supporting work in professional environments.

An important feature of LLMs is the ability to perform in-context learning—that is, to learn how to perform a task through a few given examples in the form of demonstration, without any further model training. In-context learning can follow human-like decision process, easily incorporate human knowledge and reduce computational costs [8]. As illustrated in Figure 1-1, by providing the examples, LLMs can learn and perform better than when no examples are given.

Despite these advantages, the effectiveness of in-context learning depends heavily on how examples are selected (Figure 1-2) and organised inside the prompt. Small changes made in the prompt, such as orders of in-context examples [9, 10], can result in noticeably different outputs from the model. This makes prompt engineering a complex process and requiring users to rely on trial and error for optimal results.

Moreover, while longer prompts to provide more examples might seem beneficial, it might come with complications which do not always help with better performance. Important information can introduce noise and irrelevant information when prompts

No-Shot and Two-Shot Prompting for Sentiment Analysis	
No-Shot Prompt	Two-Shot Prompt
System: Your task is to determine whether the review text is Positive or Negative. Answer with exactly one word: 'Positive' or 'Negative'. User: Product arrived on time and was the right part. The price could have been better. Answer: Negative	System: Your task is to determine whether the review text is Positive or Negative. Answer with exactly one word: 'Positive' or 'Negative'. User: Keeps drinks cold. Fairly quiet under my bar. Assistant: Positive User: does not snap securly onto flexible metal tubing Assistant: Negative User: Product arrived on time and was the right part. The price could have been better. Answer: Positive

Figure 1-1: Example of how in-context learning improves large language model output. The figure compares a no-shot prompt with a two-shot prompt for sentiment analysis, showing that the model’s prediction improved when we provided two examples. Outputs were generated using GPT-4o (version 2024-11-20) accessed via the Azure API.

are too long [11], and performance significantly degrades when the relevant information is situated in the middle [12].

1.1 Motivations

To address these challenges of in-context learning, this thesis is inspired by recent works in ensemble-based prompt strategies. A study by Tonolini et al. (2024) [13] proposes the Bayesian Prompt Ensembles (Bayesian PE) approach. In this method, predictions from multiple different prompts are aggregated using Bayesian model averaging. This method can achieve more stable and reliable results than relying on a single, lengthy prompt. In another study, Liu et al. (2024) [14] proposed a multi-prompt ensemble decoding framework, which aggregates token-level probabilities across prompt variants, to improve prediction.

Motivated by these insights, we propose a divide-and-aggregate method: instead

Impact of Different In-Context Example Sets for Sentiment Analysis	
Example Set 1	Example Set 2
System: Your task is to determine whether the review text is Positive or Negative. Answer with exactly one word: 'Positive' or 'Negative'. User: The part was delivered on time, was easy to install, and all parts worked. Assistant: Positive User: It's not the genuine part...a knock off cleverly disguised Assistant: Negative User: have to return need square ones but nice Answer: Positive	System: Your task is to determine whether the review text is Positive or Negative. Answer with exactly one word: 'Positive' or 'Negative'. User: Keeps drinks cold. Fairly quiet under my bar. Assistant: Positive User: does not snap securely onto flexible metal tubing Assistant: Negative User: have to return need square ones but nice Answer: Negative

Figure 1-2: Example of how different in-context example sets affect the outputs of the LLM. Outputs were generated using GPT-4o (version 2024-11-20) accessed via the Azure API.

of using all examples in one long prompt, we split examples into groups, each used with the same instruction and query to prompt the LLMs separately. The outputs from these groups are then aggregated to form the final output. This method aims to improve the performance of LLMs through prompt optimisation in in-context few-shot learning, using a strategy that is both simple and efficient compared to other recent methods.

1.2 Research Questions

The key research questions addressed in this thesis are:

- **RQ1:** Can dividing examples into groups, prompting separately and aggregating the outputs improve the accuracy, stability and reliability?
- **RQ2:** How can the divide-and-aggregate approach be designed to achieve optimal results which can parallel or even outperform other prompt selection and

optimisation strategies, such as Bayesian and similarity-based methods?

1.3 Contributions

This thesis makes the following contributions:

- Proposes a divide-and-aggregate method to optimise prompt techniques in in-context few-shot learning.
- Provides empirical evaluation on standard datasets (such as TweetEval, IMDB Reviews and Amazon Reviews) using popular LLMs.
- Analyses experiments of the proposed approach.

1.4 Organisation

The remainder of this thesis is organised as follows:

- **Chapter 2:** Reviews literature on LLM prompting techniques, in-context learning, and prompt optimisation strategies.
- **Chapter 3:** Describes the divide-and-aggregate methodology, including the Bayesian perspective and experimental pipeline.
- **Chapter 4:** Presents experimental setup, results, and analysis on selected datasets.
- **Chapter 5:** Discusses conclusions, implications, and potential future work.

Chapter 2

Literature Review

2.1 Prompting Techniques

It is a consensus in the research community that different prompting techniques impact the performance of LLMs on reasoning process and techniques like few-Shot learning, Chain-of-Thought (CoT) could improve the performance on various tasks [15, 16, 17, 18, 19] .

In earlier years, Brown et al. (2020), who introduced GPT-3 LLM, performed extensive experiments on GPT-3’s in-context few-shot, one-shot, and zero-shot performance across Natural Language Processing (NLP) tasks. The model achieved strong results in the few-shot setting on many NLP datasets, including translation, question-answering and common-sense reasoning. For example, GPT-3 in few-shot setting reached 86.4% accuracy on the LAMBADA word prediction dataset compared to 76.2% in zero shot setting [3].

A study by Wei et al. [20] explores the Chain-of-Thought (CoT) prompting technique to improve the reasoning abilities of Large Language Models (LLMs). The study evaluates CoT prompting on various benchmarks including GSM8K for math word problems, Data and Sports understanding datasets from BIG-bench effort for commonsense reasoning, and others for symbolic reasoning. For commonsense reasoning, they randomly selected examples from the benchmark datasets and manually composed chains of thought for them to use as few-shot exemplars. For example, they

selected the first ten examples at Sports dataset as exemplars as few-shot exemplars and report numbers on the rest of the evaluation set. The evaluation involved several large models, such as GPT-3 (various versions), LaMDA, PaLM, UL2, and Codex. CoT prompting improved the performance in commonsense reasoning tasks. For instance, PaLM 540B showed strong performance on StrategyQA with outperforming the prior state of the art (75.6% vs 69.4%) and on sports understanding (95.4% vs 84%).

Building on the advancements of CoT, Yao et al. [21] introduces the Tree of Thoughts (ToT) framework, an advancement of the CoT prompting technique. It extended the CoT approach by maintaining a tree structure of thoughts, where each node represents a partial solution or an intermediate reasoning step. For example, in the Game of 24, where the goal is to use 4 numbers and arithmetic operations (+-*/) to obtain 24, if input “4 9 10 13” is given, the ToT setup will be prompting the LM to propose some possible next steps and evaluating every thought candidate for each step. Breadth-first search (BFS) will be performed to select the best solution. According to Yao et al. , ToT significantly enhances LLM’s problem-solving abilities on three novel tasks: Game of 24, Creative Writing, and Mini Crosswords. For instance, in Game of 24, while GPT-4 with chain-of-thought prompting only solved 4% of tasks, our method achieved a success rate of 74%.

A new approach, the Graph-of-Thoughts (GoT), has also been introduced in 2023. Unlike tree-based approaches, GoT organises the reasoning process as a graph, allowing for more flexibility and multiple directions into the process. Abdelnour et al. (2023) [22] developed the GoT framework, demonstrating improvements on reducing error rates in sorting reasoning tasks compared to ToT and CoT methods.

2.2 Example Selection Strategies

While in-context learning and prompting techniques can enhance the performance of LLMs, research has shown that the choice or arrangement of examples can impact on the in-context learning outputs.

Lu et al. (2022) investigated the effect of the ordering of same examples in few-shot prompts, revealing that performance can vary. Certain ordering in some LLMs lead to improved outputs while others may result in random performance. It indicates the challenge which comes from the uncertainty of selecting the “better” orders to achieve better performance. Thus, they proposed an Entropy-based probing approach to automatically select to order of examples. Their method can achieve a 13% relative improvement for GPT-family models on eleven text classification tasks [23].

To address the uncertainty of example selection, Cai et al. (2024) propose Adaptive-Prompt. They use an adaptive strategy for selecting ICL examples using the feedback from previously chosen examples. It iteratively selects the most uncertain question at each round, annotate and add it into the example set. After expanding the coverage example set, they use the enriched example set for testing. The approach can consistently improve the accuracy on the experiments by 0.3% to 0.8% [24].

Wang et al. (2024) proposed an iterative retrieval framework, LLM-R. Their method trains a dense retriever by prompting to LLM feedback and then using a cross-encoder-based reward model. This method will help to select high-quality in-context examples for certain query. Their approach achieves a 7.8% point improvement over random selection on the experiments [25].

2.3 Calibration and Ensembling Methods

It has been shown that LLMs can perform better with “right” examples. However, selecting in-context examples often requires building extra models, as well as using larger pool of labelled examples. Other strategies have been introduced without selections on the examples. These methods typically use multiple prompts with fixed contents or examples and then calibrate or aggregate the outputs to achieve a more stable and robust final output.

To improve few-shot performance, Zhao et al. (2021) proposes a calibration method based on the model bias. They firstly estimate the bias of the model using fixed prompts without including final queries. Next, they correct the bias by

calibrating the model’s output probabilities according to the calculated bias. The final prediction is then made using the calibrated probabilities. Their contextual calibration approach achieved an improvement on GPT-3’s performance by up to 30% as well as reduce variance across different sets of prompts.

Hou et al. (2023) focus on black-box models and propose an ensemble method for text classification. In their approach, each query is processed with a small pool of prompt templates. Multiple outputs are collected. Each output is then mapped to a class label via a verbaliser, and the AdaBoost ensemble algorithm is then used to calculate and assign the weights for each prompt. The final prediction is made by aggregating the weighted votes from the classifiers. This technique achieves results that are matching or outperforming the state-of-the-art methods. For example, it delivers a 4% improvement in accuracy over the best black-box baseline on the AG’s News dataset [26].

Similarly, Tonolini et al. (2024) introduce Bayesian Prompt Ensembles (BayesPE). Their approach constructs a weighted ensemble of semantically equivalent prompts. Each prompt is formulated with a different instruction and used for the same classification task. BayesPE then learns prompt-specific weights from a small labelled validation set. At inference, the model’s final prediction is calculated by aggregating the class probabilities from all prompts using the learned weights. Extensive experiments show that BayesPE not only improves the prediction accuracy compared to several baselines, but also achieves uncertainty calibration [27].

Chapter 3

Methodology

In this chapter we introduce our proposed entropy-weighted prompt ensemble method. Our method combines outputs from multiple in-context example prompts by weighing them according to their information gain, then produces a calibrated final prediction.

3.1 Divide-and-Aggregate Approach

Our approach uses a divide-and-aggregate strategy for prompt-based predictions. For each dataset, we randomly select a pool of N labelled examples which are balanced across L classes. These examples are then randomly divided into K groups, ensuring each group is balanced across L classes.

For each test sample, we first obtain the baseline probability distribution by a no-shot prompt. Next, we construct K prompts each consisting of the test sample combined with one set of the example groups. This returns K sets of predicted class probability distributions. All these predictions are then aggregated using an entropy-weighted method, where more information gained prompts are assigned higher weights for the final output.

Additionally, we expand to a variant where the baseline prompt is constructed using M fixed examples instead of a no-shot prompt. In this setup, we generate K groups each of which includes $N - M$ additional examples. We perform the same procedure to produce the final output.

3.2 Prompt construction

In our work, we format each prompt using three roles: **system**, **user**, and **assistant**. The **system** defines the task and related instructions. Each **user** is the example input or the test query, and each **assistant** is example label or model-generated prediction for the test input. For each test sample, we use three main types of constructions: no-shot, K-shot, and a base-few-shot variant. **No-shot prompts** only consist of a system instruction followed by the user query. And the assistant is used to guide the LLMs to generate the prediction. **K-shot prompts** follow the same structure, but with K labelled example pairs listed above the query input. In the **base-few-shot variant**, the first few examples in the K-shot prompt are the same across all K groups. The general structure of prompts in our work is illustrated in Figure 3-1.

"No-Shot Prompt" vs "K-Shot Prompt"	
No-Shot Prompt	K-Shot Prompt
### System: [Task instruction] ### User: [Query input] ### Assistant:	### System: [Task instruction] ### User: [Example input 1] ### Assistant: [Label 1] ... ### User: [Example input K] ### Assistant: [Label K] ### User: [Query input] ### Assistant:

Figure 3-1: Side-by-side schematic of no-shot and K-shot prompt formats used in our experiments

3.3 Entropy-Weighted Aggregation

After obtaining all the predicted class probability distributions for each test sample in both baseline (no-shot and base-few-shot) and the K prompts, we aggregate these outputs using an entropy-weighted Bayesian approach. The overall process is as follows:

1. **Compute Prior and Posterior Probabilities:** For each test sample, we obtain a prior probability distribution P_{prior} (from a no-shot or base-few-shot prompt) and K posterior probability distributions P_i (from the K group prompts).
2. **Calculate Entropy:** We calculate the entropy for both the prior and each posterior distribution:

$$H_{\text{prior}} = - \sum_{c=1}^L P_{\text{prior}}(c) \log P_{\text{prior}}(c) \quad (3.1)$$

$$H_i = - \sum_{c=1}^L P_i(c) \log P_i(c) \quad (3.2)$$

3. **Compute Information Gain and Weights:** For each prompt, we calculate the information gain, clipped at zero, and normalise to obtain weights:

$$\beta_i = \max(0, H_{\text{prior}} - H_i) \quad (3.3)$$

$$\tilde{\beta}_i = \frac{\beta_i}{\sum_{j=1}^K \beta_j} \quad (3.4)$$

4. **Aggregate Predictions:** We first take the logarithm of all probabilities, and then compute the weighted sum of the log-likelihood differences between each posterior and the prior, adding to the prior log-probabilities:

$$\log P_{\text{agg}}(c) = \sum_{i=1}^K \tilde{\beta}_i [\log P_i(c) - \log P_{\text{prior}}(c)] + \log P_{\text{prior}}(c) \quad (3.5)$$

5. **Compute Final Prediction:** We apply the softmax function to obtain the

final probability distribution:

$$P_{\text{final}}(c) = \frac{\exp(\log P_{\text{agg}}(c))}{\sum_{c'=1}^L \exp(\log P_{\text{agg}}(c'))} \quad (3.6)$$

The predicted label is assigned as:

$$\hat{y} = \arg \max_c P_{\text{final}}(c) \quad (3.7)$$

Using this method, we calibrate the predictions, so that more informative prompts contribute more to the final result, without any additional modelling or process for example selection. The overall workflow is illustrated in Figure 3-2.

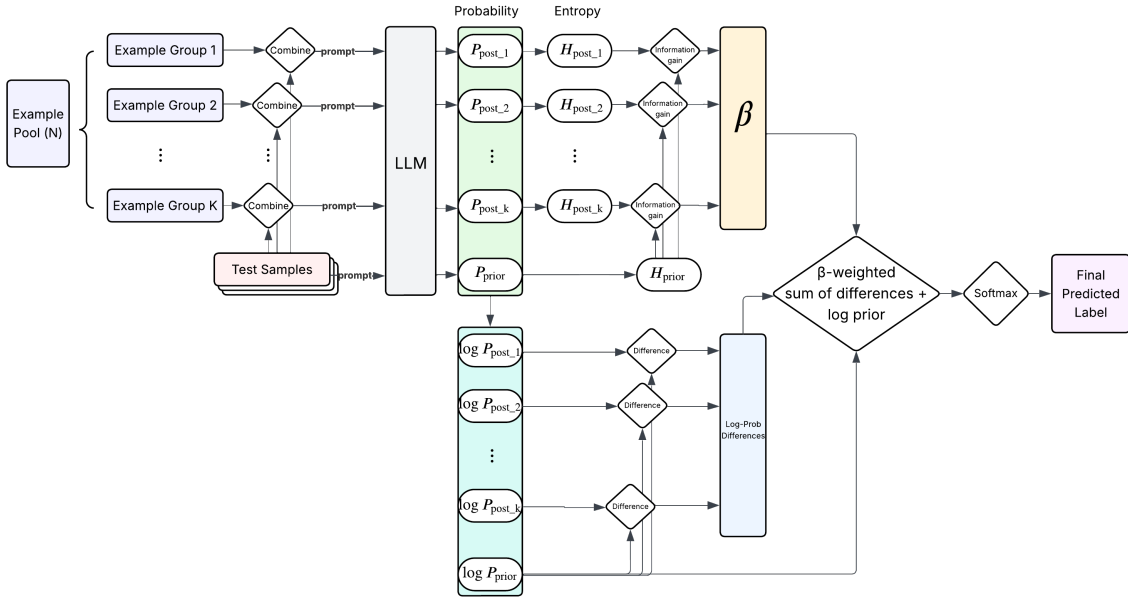


Figure 3-2: Detailed workflow of the β -weighted aggregation process.

Chapter 4

Experiments, Results, and Analysis

The following section reports the experimental results and analysis for all evaluated LLMs on datasets, followed by the metric-based comparison and further discussion.

4.1 Large Language Models

In our proposed approach, we required access to the full model output probabilities (logits) for each prediction. Therefore, all experiments were conducted using open-source LLMs that could be run locally and provide access to raw prediction scores. In addition, due to computational resource limitations, we focused only on relatively lightweight models, with maximum size of 8 billion parameters. Thus, we selected five relatively new and instruction-tuned models from different leading groups in LLM research. We tried to stretch the range of size from 1B to 8B for better coverage across ultra-lightweight and small-scale LLMs. Table 4.1 summarises the models evaluated in this work.

Table 4.1: Summary of Evaluated Large Language Models

Model Name	Developer	Size	Release Date
Gemma-3-1B-it[28]	Google	1B	Apr 2025
Llama-3.2-3B-Instruct[29]	Meta	3B	Sep 2024
Qwen2.5-7B-Instruct-1M[30]	Alibaba	7B	Jan 2025
Mistral-7B-Instruct-v0.2[31]	Mistral AI	7B	Sep 2024
Llama-3.1-8B-Instruct[32]	Meta	8B	Sep 2024

4.2 Datasets

In this work, we focus on sentiment analysis experiments with LLMs, applying our proposed methods and comparing them with other baselines. We selected two binary classification datasets – Amazon Reviews [33] and IMDB Reviews [34] as well as a more challenging three-class classification dataset. a bit harder which three class classification – TweetEval sentiment [35].

4.2.1 Amazon Reviews

The Amazon Reviews dataset contains reviews submitted by users for products. Each review is labelled with a binary sentiment (*positive* or *negative*). Considering we will use as many as ten examples in our prompt, we only retained samples with a length between 4 and 49 words to limit the length of the final queries. After filtering, we randomly constructed three mutually exclusive subsets: an in-context example pool(100 samples, further divided into 10 balanced groups at random), a validation set for Bayesian prompt ensemble weight calculation(100 samples), and a test set(500 samples). All subsets were stratified to maintain the class balance.

4.2.2 IMDB Movie Reviews

The IMDB Movie Reviews dataset comprises free-text reviews of movies. Each review is annotated with a binary sentiment label (*positive* or *negative*). Similar to our processing of the Amazon Review dataset, we retained only reviews containing fewer than 60 words to ensure limiting the final length of prompts. And then, an in-context example pool (100 samples, further divided into 10 balanced groups at random), a validation set for Bayesian prompt ensemble weight calculation (100 samples), and a test set (500 samples) are randomly constructed. All subsets were stratified to maintain balanced on sentiment classes.

4.2.3 TweetEval Sentiment

The TweetEval Sentiment dataset consists of tweet texts which are annotated with one of three sentiment classes: *positive*, *neutral*, or *negative*. This dataset includes the informal language and social media style expressions, making it a more challenging multiclass classification task. For consistency with our experimental framework, we constructed three mutually exclusive subsets: an in-context example pool (90 samples, divided into 10 groups with balanced classes at random), a validation set for Bayesian prompt ensemble weight calculation (120 samples), and a test set (510 samples). All splits were stratified by sentiment classes.

4.3 Experiment Setup

To evaluate our proposed approach, we conducted a total of 90 experiment runs. For each combination of five LLMs and three benchmark datasets, we performed two pipelines: (1) the **standard no-shot and K-shot** process, and (2) the **Fixed-base few-shot with extended K-shot** process (as detailed below). Each run returns the predicted class probabilities for each prompt.

4.3.1 Standard No-shot and K-shot Process

For each model and dataset combination, the first pipeline consists of three main steps:

No-shot: For each test sample, the model is prompted using only the test query, without any in-context examples.

K-shot: For each test sample, we construct 10 prompts, each with a different pre-assigned group of in-context examples (groups are balanced across classes). The model is prompted with each of these 10 prompts, and the predicted class probabilities are collected.

Bayesian weight calculation: For the comparison methodology Bayesian PE, we obtain the weights to assign to each group. This is performed on a separate

validation set (not overlapping with the test set). We construct 10 prompts per sample, each using the same groups as in the K-shot setup. The model is prompted with each of these 10 prompts, and the predicted class probabilities are collected.

4.3.2 Fixed-base few-shot with extended K-shot

While pipeline one no-shot versus few-shot comparison is a common setup in the community, the difference in performance can be dominated by presence or absence of in-context examples. Thus, in this pipeline, we implement a fixed-base setup and then increase the numbers of shots to K . This approach could provide us a fuller picture of how assembling with additional examples contributes to the final predictions. For each model and dataset combination, the second pipeline consists of:

Fixed-base few-shot: For each test sample, the model is prompted with a fixed set of base in-context examples (e.g., 4 examples), along with the test query. The predicted class probabilities are collected.

Extended K-shot: For each test sample, we construct 10 prompts. Each prompt includes the same fixed base examples as above, plus a different group of additional in-context examples (e.g., 6 per group, groups are balanced across classes). The model is prompted with each of these 10 prompts, and the predicted class probabilities are collected.

Bayesian weight calculation: Similar to pipeline one, we perform queries on a separate validation set (not overlapping with the test set). For each validation sample, we construct 10 prompts, each with the fixed base examples and one of the additional groups (same as in the Extended K-shot setup). The predicted class probabilities are collected.

4.4 Baselines and Metrics

4.4.1 Baselines

To perform thorough evaluation on our proposed approach, we select several baselines for comparison and analysis. In both pipelines, we construct the following baselines for each model and dataset combination.

No-shot / Base-few-shot: As a starting point, the model is either prompted only with the test query without in-context examples (no-shot), or with a fixed set of base in-context examples (base few-shot). The results from these provide a basic observation on the fundamental ability of the LLM to perform the task on each dataset.

Bayesian Prompt Ensemble (BPE): Following Tonolini et al. (2024) [13], we aggregate predictions from multiple prompts, using weights learned on a separate validation set. Unlike the original BPE method, which aggregates prompts with different instructions, our approach keeps a universal instruction and only varies the in-context example groups.

Best group: We identify the group of in-context examples that achieves the lowest Negative Log-Likelihood (NLL) on the validation set. We then only evaluate outputs from this same group on the test set and report all metrics.

Worst group: Similarly, we identify the group with the highest NLL on the validation set, and use only this group’s outputs for evaluating and reporting metrics on the test set.

These baselines provide a comparable foundation for assessing the contributions of in-context example selection, prompt ensembling, and calibration in our experiments.

4.4.2 Metrics

For each baseline and our proposed method, we calculate multiple metrics following Tonolini et al. (2024) [13]. These metrics cover both classification performance and probabilistic calibration:

- **Negative Log-Likelihood (NLL):** Measures the difference between the predicted probabilities and the true labels; lower is better.
- **ROC-AUC (AUC):** Area under the Receiver Operating Characteristic curve; higher is better.
- **Expected Calibration Error (ECE):** Weighted average of the absolute difference between the average accuracy and average confidence across different confidence bins; lower is better.
- **Maximum Calibration Error (MCE):** The largest gap between model confidence and accuracy among all bins; lower is better.
- **Brier score:** Mean squared error between the predicted probabilities and the true labels; lower is better.
- **Macro-F1:** The unweighted average F1 score across all classes; higher is better.
- **Accuracy:** The proportion of correct predictions; higher is better.

All metrics are computed on the test set for each model and dataset combination.

4.5 Results

Across two experimental pipelines, three datasets, five models, and seven evaluation metrics, we collected a full set of results for each combination. We observed our method consistently achieves strong performance in both Accuracy and MacroF1, often matching or exceeding established baselines on the sentiment classification tasks. Figure 4-1 presents direct comparisons of MacroF1 scores on each model and dataset under both pipelines. Full numerical results for all metrics, models, datasets, and methods are provided in Appendix Tables A.1–A.6.

4.6 Analysis

4.6.1 Accuracy-Related Performance

Across the various datasets and models, our proposed method demonstrates competitive performance on accuracy-related metrics—including Accuracy, MacroF1, AUC, Negative Log-Likelihood (NLL), and Brier score. For the sentiment classification tasks, including both binary (Amazon and IMDB) and three-class (TweetEval) datasets, our approach often matches or closely follows the best-performing baseline, which is typically the BayesPE method. In several models and pipelines, our method even surpasses all baselines, particularly in MacroF1 or Accuracy.

BayesPE does show an overall advantage in NLL and Brier score, reflecting its calibration-focused design. Our results show more variability on these metrics: in some cases, our method is close to BayesPE within a small margin, while in others it does not perform as strongly. However, our approach generally outperforms the WorstGroup baseline as well as the no-shot or base prompting in nearly all scenarios.

4.6.2 Calibration-Related Performance

While our method demonstrates competitive performance on accuracy across tasks, it returns variable results on calibration-related metrics, such as ECE and MCE, in different scenarios. In nearly all datasets and models, the BayesPE baseline achieves the lowest ECE and MCE values. This is expected, as BayesPE is designed to optimise probabilistic calibration through likelihood-based weights. In contrast, our method is focusing on information gain and dynamic aggregation based on change of confidence level, which can lead to higher ECE and MCE scores.

Figure 4-2 summarises the comparisons on ECE scores, illustrating our method’s calibration performance can vary depending on the dataset and scenario.

4.6.3 Cross-Setting Performance

To better understand the efficiency of our method, we evaluate its performance across the no-shot and base-few-shot pipelines. First, we observe that our method improves performance in both settings, though—as expected—it achieves greater improvement in the no-shot pipeline. This confirms that the gains are not solely due to in-context examples but also stem from our method. Second, in most scenarios, our method outperforms at least one of the baselines, either the “worst group” or “best group,” suggesting that the improvement is not simply from using more examples. Finally, when compared to BayesPE, our method remains competitive. While BayesPE often achieves better calibration, our method offers nearly matched or sometimes even stronger accuracy performance.

We also expect that moving from binary to three-class classification tasks will become more challenging for all methods. Nevertheless, our method remains competitive with BayesPE without any significant performance drops. Overall

4.6.4 Cross-Model Performance

Across all experimental scenarios, our method consistently improves accuracy, MacroF1, and AUC over the fundamental baseline (No-shot or Base-Few-Shot). The scale of these gains is similar across all larger-size models in our scope (Qwen 7B, Llama 8B, Llama3b, Mistral 7B), while for the small-capacity model Gemma, the improvements are less stable and show fluctuations. A similar pattern is observed for the BayesPE method. Overall, these results indicate that the effectiveness of ensemble aggregation methods is not specific to any particular models, but can provide generalisable improvements to different LLMs.

4.7 Discussion

Our experimental results and analysis demonstrate the effectiveness of the proposed entropy-weighted aggregation method for in-context learning with LLMs. Across

all scenarios, our method consistently improves upon standard prompting strategies and closely matches the performance of the BayesPE method, especially in terms of accuracy, MacroF1, and AUC for both binary and multiclass sentiment tasks. These improvements are general across all evaluated LLMs, supporting the broad applicability of our approach.

A key advantage of our method is its simplicity and practicality: it requires no additional modelling of example selections and does not depend on extra labelled data for calibration, unlike BayesPE. Moreover, our aggregation is dynamic, based on individual sample rather than applying fixed weights to all predictions. These features make our approach a practical option for the in-context learning community.

Limitations: This work is subject to several limitations. **First**, all experiments were conducted on open-source models with up to 8B parameters; the method’s behaviour with larger models remains to be explored. **Second**, our evaluations solely focused on sentiment analysis tasks, so effectiveness on other NLP tasks is not yet tested. **Third**, all classification problems covered were either binary or three-class; tasks involving a larger number of classes or more complex label structures should be considered for future work.

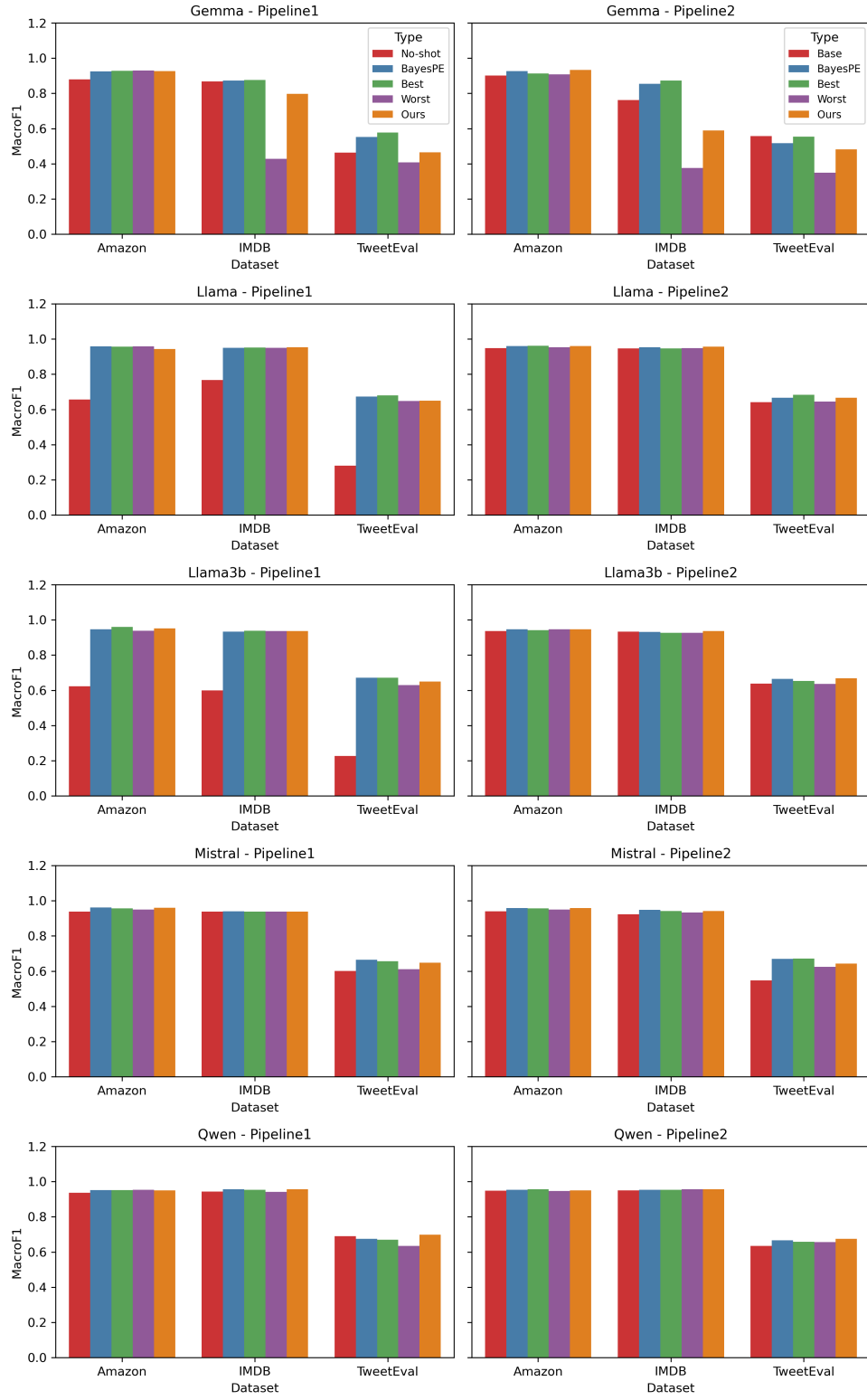


Figure 4-1: MacroF1 scores for each model and dataset, across both pipelines. For each chart, bars represent the different methods for each dataset.

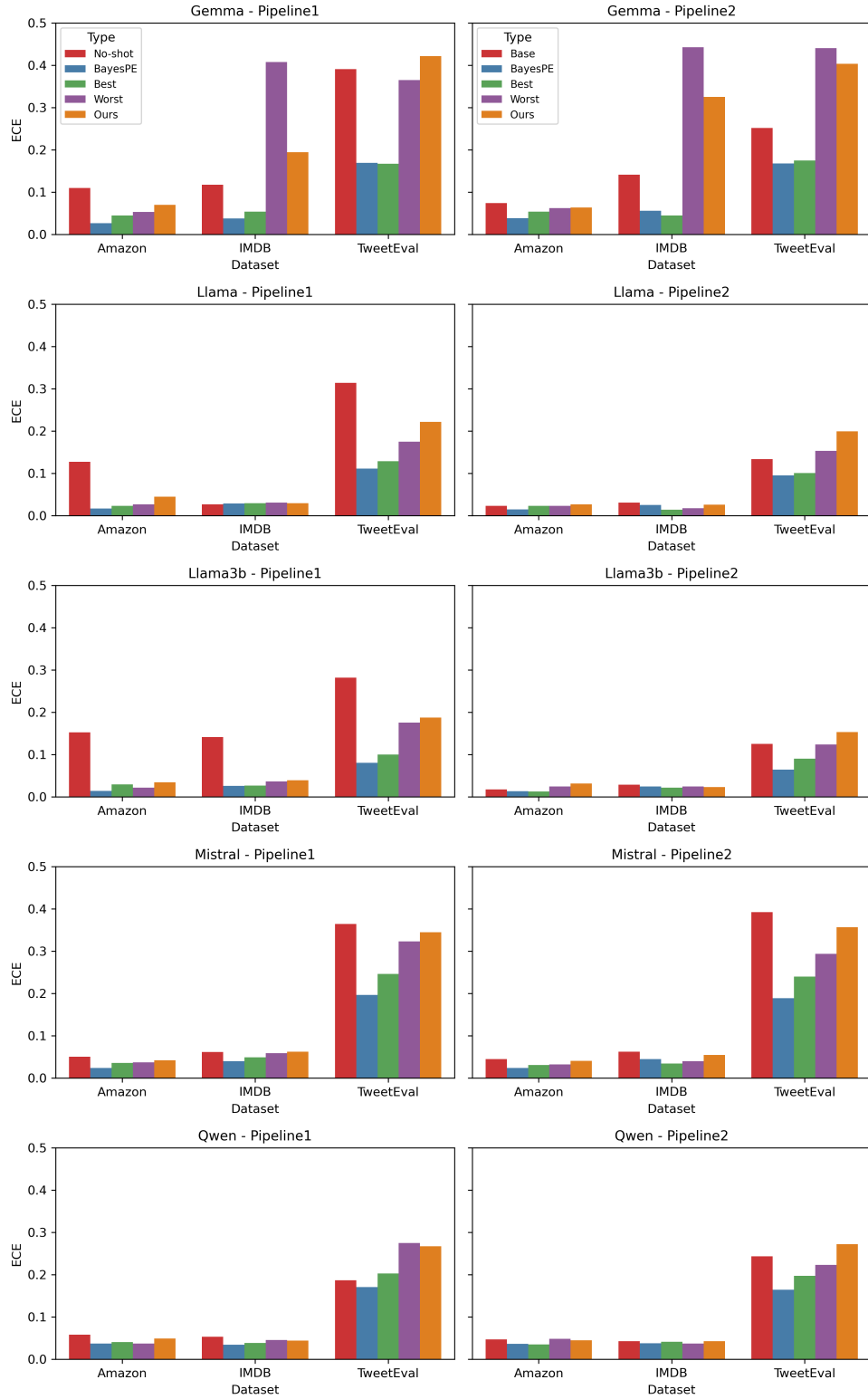


Figure 4-2: ECE scores for each model and dataset, across both pipelines. For each chart, bars represent the different methods for each dataset.

Chapter 5

Conclusion

In this work, we proposed a divide-and-aggregate approach for prompt engineering in in-context few-shot learning with large language models (LLMs). Motivated by the limitations of simple prompt techniques and complexity of existing prompt construction and selection strategies, we introduced a method that dynamically combines outputs from randomly divided groups of examples. Our goal was to improve the reliability, stability, and accuracy of LLM predictions without requiring additional labelled data or complex processes of prompt construction or selection modelling.

Through experiments on three sentiment analysis datasets using five open-source instruction-tuned LLMs with up to 8B parameters, our approach matched or outperformed traditional baseline approaches in key metrics such as Accuracy, MacroF1, and AUC, and remained competitive with the state-of-the-art Bayesian Prompt Ensemble (BayesPE) method. Although our method showed variability in calibration metrics (ECE, MCE) compared to calibration-focused baselines like BayesPE, it achieved these results without relying on additional models or labelled validation data, making it a flexible and practical alternative for prompt ensembling.

This work is not without limitations. Our experiments focused on sentiment analysis with binary and three-class problems and used lighter-weight LLMs up to 8B parameters. The effectiveness of this approach for larger models, more complex NLP tasks, and more diverse label structures remains an open question.

Looking forward, this research will continue and expand. Future work will explore

applying the divide-and-aggregate strategy to broader range of NLP tasks, investigate ways to further improve probabilistic calibration, and evaluate the method on larger LLMs. The aggregation process will also be reviewed and refined to better adapt to different types of scenarios. With the foundations established in this thesis, we aim to further enrich prompt engineering techniques and contribute simpler but effective solutions for in-context learning with LLMs.

Appendix A

Tables

Table A.1: Results on Amazon Reviews Dataset (Pipeline 1)

Model	Method	NLL	AUC	ECE	MCE	Brier	MacroF1	Accuracy
Gemma								
	No-shot	0.6881	0.9561	0.1098	0.6724	0.1059	0.8791	0.880
	BayesPE	0.2233	0.9712	0.0269	0.1607	0.0603	0.9240	0.924
	Best	0.2508	0.9684	0.0446	0.3016	0.0609	0.9280	0.928
	Worst	0.3479	0.9688	0.0534	0.5254	0.0636	0.9299	0.930
	Ours	0.5705	0.9602	0.0704	0.6354	0.0694	0.9259	0.926
Llama								
	No-shot	0.5581	0.9520	0.1274	0.3957	0.1899	0.6563	0.686
	BayesPE	0.1269	0.9885	0.0170	0.2253	0.0337	0.9580	0.958
	Best	0.1504	0.9841	0.0234	0.2996	0.0362	0.9560	0.956
	Worst	0.1359	0.9881	0.0264	0.1492	0.0356	0.9580	0.958
	Ours	0.2071	0.9828	0.0450	0.7128	0.0496	0.9440	0.944
Llama3b								
	No-shot	0.5608	0.9578	0.1527	0.4671	0.1965	0.6235	0.664
	BayesPE	0.1497	0.9841	0.0139	0.1358	0.0405	0.9460	0.946
	Best	0.2026	0.9783	0.0291	0.3790	0.0387	0.9600	0.960
	Worst	0.1650	0.9837	0.0216	0.2658	0.0452	0.9379	0.938
	Ours	0.1938	0.9808	0.0340	0.4145	0.0438	0.9520	0.952
Mistral								
	No-shot	1.0164	0.9574	0.0507	0.6937	0.0560	0.9380	0.938
	BayesPE	0.2570	0.9872	0.0241	0.2163	0.0303	0.9620	0.962
	Best	0.4628	0.9823	0.0359	0.5833	0.0364	0.9560	0.956
	Worst	0.3976	0.9833	0.0369	0.3052	0.0430	0.9500	0.950
	Ours	0.9758	0.9642	0.0423	0.4596	0.0407	0.9600	0.960
Qwen								
	No-shot	0.3928	0.9782	0.0584	0.6258	0.0558	0.9359	0.936
	BayesPE	0.1847	0.9880	0.0372	0.6322	0.0392	0.9520	0.952
	Best	0.2143	0.9846	0.0405	0.7123	0.0412	0.9520	0.952
	Worst	0.2080	0.9853	0.0369	0.7249	0.0386	0.9540	0.954
	Ours	0.3631	0.9807	0.0492	0.9255	0.0477	0.9500	0.950

Table A.2: Results on Amazon Reviews Dataset (Pipeline 2)

Model	Method	NLL	AUC	ECE	MCE	Brier	MacroF1	Accuracy
Gemma								
	Base	0.3492	0.9676	0.0743	0.6304	0.0766	0.9020	0.902
	BayesPE	0.2796	0.9732	0.0383	0.2232	0.0634	0.9258	0.926
	Best	0.3491	0.9706	0.0537	0.1675	0.0742	0.9137	0.914
	Worst	0.4067	0.9668	0.0625	0.4834	0.0798	0.9075	0.908
	Ours	0.3909	0.9679	0.0638	0.4127	0.0652	0.9339	0.934
Llama								
	Base	0.1528	0.9836	0.0229	0.3692	0.0377	0.9480	0.948
	BayesPE	0.1260	0.9868	0.0149	0.3099	0.0311	0.9600	0.960
	Best	0.1304	0.9868	0.0233	0.5553	0.0319	0.9620	0.962
	Worst	0.1353	0.9866	0.0233	0.5000	0.0350	0.9540	0.954
	Ours	0.1469	0.9858	0.0268	0.6762	0.0342	0.9600	0.960
Llama3b								
	Base	0.1632	0.9820	0.0175	0.1826	0.0468	0.9360	0.936
	BayesPE	0.1528	0.9822	0.0135	0.5273	0.0411	0.9460	0.946
	Best	0.1621	0.9819	0.0125	0.6066	0.0434	0.9419	0.942
	Worst	0.1636	0.9822	0.0247	0.4254	0.0447	0.9460	0.946
	Ours	0.1670	0.9836	0.0317	0.6318	0.0441	0.9460	0.946
Mistral								
	Base	0.4204	0.9823	0.0451	0.4664	0.0473	0.9400	0.940
	BayesPE	0.3067	0.9856	0.0241	0.5145	0.0321	0.9580	0.958
	Best	0.4118	0.9831	0.0307	0.4662	0.0360	0.9560	0.956
	Worst	0.3888	0.9839	0.0323	0.2107	0.0415	0.9500	0.950
	Ours	0.5737	0.9796	0.0406	0.4320	0.0409	0.9580	0.958
Qwen								
	Base	0.2544	0.9826	0.0466	0.5118	0.0469	0.9480	0.948
	BayesPE	0.1987	0.9854	0.0367	0.3787	0.0401	0.9540	0.954
	Best	0.2037	0.9852	0.0353	0.4961	0.0401	0.9560	0.956
	Worst	0.2536	0.9840	0.0483	0.5738	0.0482	0.9459	0.946
	Ours	0.2695	0.9839	0.0450	0.7811	0.0462	0.9500	0.950

Table A.3: Results on IMDB Reviews Dataset (Pipeline 1)

Model	Method	NLL	AUC	ECE	MCE	Brier	MacroF1	Accuracy
Gemma								
	No-shot	0.6517	0.9540	0.1179	0.5935	0.1165	0.8674	0.868
	BayesPE	0.3595	0.9643	0.0380	0.3415	0.1019	0.8730	0.874
	Best	0.3593	0.9596	0.0541	0.2125	0.0963	0.8755	0.876
	Worst	2.0119	0.9452	0.4080	0.7869	0.4031	0.4281	0.546
	Ours	1.2646	0.9576	0.1942	0.5736	0.1932	0.7977	0.804
Llama								
	No-shot	0.4560	0.9526	0.0266	0.0613	0.1496	0.7678	0.778
	BayesPE	0.1490	0.9878	0.0288	0.3003	0.0396	0.9500	0.950
	Best	0.1628	0.9857	0.0297	0.3656	0.0407	0.9520	0.952
	Worst	0.1634	0.9859	0.0310	0.5224	0.0417	0.9500	0.950
	Ours	0.1654	0.9862	0.0298	0.5175	0.0407	0.9540	0.954
Llama3b								
	No-shot	0.5545	0.9473	0.1415	0.3153	0.1936	0.5991	0.646
	BayesPE	0.1759	0.9801	0.0262	0.4209	0.0478	0.9340	0.934
	Best	0.1889	0.9792	0.0268	0.3805	0.0472	0.9380	0.938
	Worst	0.2058	0.9801	0.0367	0.5136	0.0532	0.9359	0.936
	Ours	0.1976	0.9803	0.0390	0.7029	0.0539	0.9360	0.936
Mistral								
	No-shot	1.3758	0.9488	0.0619	0.6313	0.0589	0.9380	0.938
	BayesPE	0.4221	0.9809	0.0397	0.2813	0.0490	0.9399	0.940
	Best	0.8939	0.9661	0.0493	0.5779	0.0494	0.9379	0.938
	Worst	1.0555	0.9600	0.0586	0.4618	0.0551	0.9379	0.938
	Ours	1.5459	0.9488	0.0621	0.0832	0.0617	0.9380	0.938
Qwen								
	No-shot	0.3618	0.9773	0.0536	0.9159	0.0523	0.9440	0.944
	BayesPE	0.2149	0.9805	0.0343	0.2315	0.0403	0.9560	0.956
	Best	0.2234	0.9821	0.0389	0.6225	0.0414	0.9540	0.954
	Worst	0.2804	0.9815	0.0455	0.5872	0.0465	0.9420	0.942
	Ours	0.3522	0.9804	0.0444	0.1645	0.0438	0.9560	0.956

Table A.4: Results on IMDB Reviews Dataset (Pipeline 2)

Model	Method	NLL	AUC	ECE	MCE	Brier	MacroF1	Accuracy
Gemma								
	Base	0.7001	0.9491	0.1412	0.3310	0.1804	0.7627	0.774
	BayesPE	0.4024	0.9683	0.0559	0.1285	0.1120	0.8540	0.856
	Best	0.3494	0.9659	0.0452	0.1507	0.0954	0.8734	0.874
	Worst	2.2029	0.9544	0.4428	0.7550	0.4306	0.3763	0.520
	Ours	1.6261	0.9568	0.3255	0.7749	0.3250	0.5894	0.642
Llama								
	Base	0.1656	0.9836	0.0309	0.1948	0.0434	0.9460	0.946
	BayesPE	0.1452	0.9872	0.0250	0.2968	0.0392	0.9540	0.954
	Best	0.1479	0.9868	0.0140	0.1820	0.0415	0.9460	0.946
	Worst	0.1527	0.9840	0.0178	0.5078	0.0409	0.9480	0.948
	Ours	0.1511	0.9863	0.0259	0.4418	0.0384	0.9560	0.956
Llama3b								
	Base	0.2010	0.9742	0.0289	0.2061	0.0559	0.9340	0.934
	BayesPE	0.1834	0.9780	0.0247	0.3776	0.0499	0.9320	0.932
	Best	0.1982	0.9755	0.0216	0.2477	0.0552	0.9258	0.926
	Worst	0.2140	0.9744	0.0248	0.1598	0.0601	0.9260	0.926
	Ours	0.1854	0.9777	0.0233	0.4901	0.0513	0.9360	0.936
Mistral								
	Base	0.8080	0.9668	0.0624	0.7438	0.0650	0.9239	0.924
	BayesPE	0.4140	0.9763	0.0447	0.4357	0.0486	0.9480	0.948
	Best	0.4595	0.9752	0.0341	0.2675	0.0488	0.9420	0.942
	Worst	0.4904	0.9666	0.0402	0.4724	0.0559	0.9339	0.934
	Ours	0.9796	0.9627	0.0549	0.4006	0.0571	0.9419	0.942
Qwen								
	Base	0.2549	0.9794	0.0430	0.6478	0.0443	0.9500	0.950
	BayesPE	0.1986	0.9807	0.0380	0.5091	0.0413	0.9540	0.954
	Best	0.2251	0.9813	0.0416	0.3764	0.0427	0.9540	0.954
	Worst	0.2113	0.9787	0.0374	0.4728	0.0400	0.9560	0.956
	Ours	0.2603	0.9810	0.0424	0.8149	0.0423	0.9560	0.956

Table A.5: Results on TweetEval Dataset (Pipeline 1)

Model	Method	NLL	AUC	ECE	MCE	Brier	MacroF1	Accuracy
Gemma								
	No-shot	2.3221	0.7953	0.3911	0.5017	0.8228	0.4628	0.531
	BayesPE	1.0486	0.8125	0.1693	0.2903	0.5781	0.5518	0.590
	Best	1.0299	0.8005	0.1672	0.2792	0.5613	0.5781	0.602
	Worst	1.7246	0.7672	0.3649	0.5545	0.8204	0.4083	0.473
	Ours	2.4234	0.7984	0.4219	0.5876	0.8580	0.4657	0.531
Llama								
	No-shot	1.6797	0.6472	0.3138	0.4638	0.8344	0.2809	0.390
	BayesPE	0.7910	0.8408	0.1111	0.4178	0.4552	0.6727	0.680
	Best	0.8560	0.8383	0.1284	0.1775	0.4694	0.6799	0.684
	Worst	0.9600	0.8281	0.1747	0.2993	0.5168	0.6483	0.655
	Ours	1.0144	0.8256	0.2215	0.4342	0.5439	0.6498	0.659
Llama3b								
	No-shot	1.3573	0.6378	0.2816	0.4474	0.7933	0.2271	0.351
	BayesPE	0.7894	0.8411	0.0808	0.1769	0.4525	0.6720	0.675
	Best	0.8104	0.8396	0.1002	0.2129	0.4516	0.6715	0.675
	Worst	1.0106	0.8262	0.1757	0.2681	0.5346	0.6290	0.637
	Ours	0.9321	0.8294	0.1872	0.3802	0.5181	0.6494	0.657
Mistral								
	No-shot	6.2672	0.7823	0.3644	0.5863	0.7475	0.6011	0.602
	BayesPE	1.3034	0.8400	0.1963	0.4626	0.5327	0.6647	0.659
	Best	2.0293	0.8238	0.2461	0.3467	0.5834	0.6557	0.649
	Worst	4.0247	0.8043	0.3234	0.4092	0.6844	0.6114	0.608
	Ours	6.6646	0.7710	0.3451	0.6143	0.6960	0.6485	0.645
Qwen								
	No-shot	1.0424	0.8543	0.1867	0.4453	0.4814	0.6903	0.696
	BayesPE	1.0050	0.8568	0.1706	0.2307	0.4933	0.6741	0.671
	Best	1.0973	0.8467	0.2028	0.3143	0.5321	0.6698	0.665
	Worst	1.4800	0.8438	0.2746	0.3755	0.6058	0.6351	0.631
	Ours	1.4473	0.8547	0.2669	0.5177	0.5571	0.6977	0.694

Table A.6: Results on TweetEval Dataset (Pipeline 2)

Model	Method	NLL	AUC	ECE	MCE	Brier	MacroF1	Accuracy
Gemma								
	Base	1.4178	0.7828	0.2522	0.3807	0.6622	0.5575	0.575
	BayesPE	1.0144	0.7890	0.1678	0.2537	0.5841	0.5168	0.553
	Best	1.0210	0.7602	0.1751	0.2708	0.5863	0.5537	0.561
	Worst	1.9552	0.7767	0.4407	0.6491	0.9177	0.3500	0.437
	Ours	1.8153	0.7922	0.4034	0.4934	0.8363	0.4824	0.522
Llama								
	Base	0.8707	0.8292	0.1334	0.2851	0.4835	0.6417	0.641
	BayesPE	0.7614	0.8434	0.0952	0.3788	0.4455	0.6665	0.669
	Best	0.8032	0.8368	0.1007	0.2380	0.4629	0.6829	0.680
	Worst	0.8833	0.8349	0.1532	0.2835	0.4959	0.6440	0.643
	Ours	0.9411	0.8380	0.1992	0.3726	0.5124	0.6657	0.669
Llama3b								
	Base	0.9014	0.8219	0.1251	0.3709	0.4987	0.6386	0.645
	BayesPE	0.7749	0.8321	0.0642	0.1070	0.4540	0.6647	0.665
	Best	0.8242	0.8212	0.0904	0.1873	0.4756	0.6535	0.655
	Worst	0.9016	0.8233	0.1240	0.2874	0.5038	0.6366	0.645
	Ours	0.9046	0.8287	0.1533	0.3647	0.4901	0.6684	0.673
Mistral								
	Base	5.3187	0.7935	0.3925	0.6306	0.8220	0.5469	0.561
	BayesPE	1.3852	0.8310	0.1891	0.4369	0.5280	0.6691	0.663
	Best	2.3700	0.8294	0.2401	0.5495	0.5559	0.6711	0.669
	Worst	3.2981	0.8097	0.2938	0.4667	0.6329	0.6248	0.618
	Ours	5.8985	0.7670	0.3568	0.8396	0.7152	0.6437	0.637
Qwen								
	Base	1.2518	0.8458	0.2436	0.3878	0.5502	0.6343	0.633
	BayesPE	0.9333	0.8506	0.1647	0.4321	0.4983	0.6656	0.661
	Best	1.0532	0.8474	0.1973	0.3828	0.5290	0.6576	0.655
	Worst	1.2369	0.8462	0.2231	0.3634	0.5654	0.6566	0.651
	Ours	1.3671	0.8508	0.2718	0.6504	0.5731	0.6744	0.671

Bibliography

- [1] T. Zhang, F. Ladhak, E. Durmus, P. Liang, K. McKeown, and T. B. Hashimoto. Benchmarking large language models for news summarization. *Transactions of the Association for Computational Linguistics*, 12:39–57, 2024.
- [2] W. Jiao, W. Wang, J. T. Huang, X. Wang, S. Shi, and Z. Tu. Is chatgpt a good translator? yes with gpt-4 as the engine. *arXiv preprint arXiv:2301.08745*, 2023.
- [3] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [4] Francesco Tonolini, Jordan Massiah, Nikolaos Aletras, and Gabriella Kazai. Bayesian prompt ensembles: Model uncertainty estimation for black-box large language models. *Amazon Science Publications*, 2024.
- [5] Juyong Jiang, Fan Wang, Jiasi Shen, Sungju Kim, and Sunghun Kim. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515*, 2024.
- [6] Ruibin Yuan, Hanfeng Lin, Yi Wang, Zeyue Tian, Shangda Wu, Tianhao Shen, Ge Zhang, Yuhang Wu, Cong Liu, Ziya Zhou, Liumeng Xue, Ziyang Ma, Qin Liu, Tianyu Zheng, Yizhi Li, Yinghao Ma, Yiming Liang, Xiaowei Chi, Ruibo Liu, Zili Wang, Chenghua Lin, Qifeng Liu, Tao Jiang, Wenhao Huang, Wenhui Chen, Jie Fu, Emmanouil Benetos, Gus Xia, Roger Dannenberg, Wei Xue, Shiyin Kang, and Yike Guo. Chatmusician: Understanding and generating music intrinsically with llm. In *Findings of the Association for Computational Linguistics: ACL 2024*, Bangkok, Thailand, August 2024.
- [7] Ran Zhang and Steffen Eger. Llm-based multi-agent poetry generation in non-cooperative environments. *arXiv preprint arXiv:2409.03659*, 2024.
- [8] Qingxiu Dong, Lei Li, Damai Dai, Ce Zheng, Jingyuan Ma, Rui Li, Heming Xia, Jingjing Xu, Zhiyong Wu, Tianyu Liu, Baobao Chang, Xu Sun, Lei Li, and

- Zhifang Sui. A survey on in-context learning. *arXiv preprint arXiv:2301.00234*, 2023.
- [9] Yanzheng Xiang, Hanqi Yan, Lin Gui, and Yulan He. Addressing order sensitivity of in-context demonstration examples in causal language models. *arXiv preprint arXiv:2402.15637*, 2024.
 - [10] Tai Nguyen and Eric Wong. In-context example selection with influences. *arXiv preprint arXiv:2302.11042*, 2023.
 - [11] Yuxiang Zhou, Jiazheng Li, Yanzheng Xiang, Hanqi Yan, Lin Gui, and Yulan He. The mystery of in-context learning: A comprehensive survey on interpretation and analysis. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 14365–14378, Miami, Florida, USA, 2024. Association for Computational Linguistics.
 - [12] Nelson F. Liu, David Dohan, Jakob Weber, Daniel Khashabi, Kelvin Guu, and Douwe Kiela. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023.
 - [13] Francesco Tonolini, Jordan Massiah, Nikolaos Aletras, and Gabriella Kazai. Bayesian prompt ensembles: Model uncertainty estimation for black-box large language models. *Amazon Science*, 2024.
 - [14] Zhengxiao Liu, Yuxian Gu, Hang Yan, Yutai Hou, Tianyi Tang, Bo Wang, Ruoxi Shang, Zeqi Lin, Yang Liu, and Maosong Sun. M-ped: Multi-prompt ensemble decoding for large language models. *arXiv preprint arXiv:2412.18299*, 2024.
 - [15] O. Press, M. Zhang, S. Min, L. Schmidt, N. A. Smith, and M. Lewis. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*, 2022.
 - [16] N. Scherrer, C. Shi, A. Feder, and D. Blei. Evaluating the moral beliefs encoded in llms. *Advances in Neural Information Processing Systems*, 36, 2024.
 - [17] R. Schaeffer, B. Miranda, and S. Koyejo. Are emergent abilities of large language models a mirage? *Advances in Neural Information Processing Systems*, 36, 2024.
 - [18] J. Sun, W. Huang, J. Wu, C. Gu, W. Li, S. Zhang, and C. He. Benchmarking chinese commonsense reasoning of llms: From chinese-specifics to reasoning-memorization correlations. *arXiv preprint arXiv:2403.14112*, 2024.
 - [19] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, and J. R. Wen. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.
 - [20] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, and D. Zhou. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

- [21] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2024.
- [22] Yuzhuo Li, Xiang Li, Yanlin Wang, Han Liu, Wei Guo, Qun Zhang, Xiangliang Zhang, and Xin Wang. Graph of thoughts: Solving elaborate problems with large language models. *arXiv preprint arXiv:2308.09687*, 2023.
- [23] Xuezhi Lu, Sewon Min, Keith Hall, et al. Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. In *Proceedings of ACL 2022*, pages 2731–2745, 2022.
- [24] Shuzhang Cai, Twumasi Mensah-Boateng, Xander Kuksov, Jing Yuan, and Shaojie Tang. The power of adaptation: Boosting in-context learning through adaptive prompting. *arXiv preprint arXiv:2412.17891*, 2024.
- [25] Liang Wang, Nan Yang, and Furu Wei. Learning to retrieve in-context examples for large language models. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1752–1767, 2024.
- [26] Tony Z. Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. Calibrate before use: Improving few-shot performance of language models. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139, pages 12697–12706. PMLR, 2021.
- [27] Francesco Tonolini, Jordan Massiah, Nikolaos Aletras, and Gabriella Kazai. Bayesian prompt ensembles: Model uncertainty estimation for black-box large language models. *arXiv preprint arXiv:2412.18299*, 2024.
- [28] Google DeepMind. Gemma 3 1b instruction-tuned model. <https://huggingface.co/google/gemma-3-1b-it>, 2025. Accessed: 2025-06-01.
- [29] Inc. Meta Platforms. Llama 3.2 3b instruct model. <https://huggingface.co/meta-llama/Llama-3.2-3B-Instruct>, 2024. Accessed: 2025-06-01.
- [30] Qwen Team. Qwen2.5-7b-instruct-1m: Long-context instruction-tuned language model. <https://huggingface.co/Qwen/Qwen2.5-7B-Instruct-1M>, 2025. Accessed: 2025-06-01.
- [31] Mistral AI. Mistral-7b-instruct-v0.2: Instruction-tuned language model. <https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.2>, 2023. Accessed: 2025-06-01.
- [32] Inc. Meta Platforms. Meta-llama-3.1-8b-instruct. <https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>, 2024. Accessed: 2025-06-01.

- [33] Amazon Web Services. Bayesian prompt ensembles: Model uncertainty estimation for black-box large language models. <https://github.com/amzn/BayesPE>, 2023. Accessed: 2025-04-08.
- [34] Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. Learning word vectors for sentiment analysis. *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 142–150, 2011. IMDB Movie Review Dataset.
- [35] Francesco Barbieri, Jose Camacho-Collados, Luis Espinosa-Anke, and Leonardo Neves. Tweeteval: Unified benchmark and comparative evaluation for tweet classification. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1644–1650, 2020. TweetEval Sentiment Dataset.